

AEGIS: Orchestrated Multi-Agent Intelligence with GraphRAG

One-line Pitch: *An orchestrator-led AI “team” that jointly reasons over diverse data sources, self-challenges its own conclusions, and quantifies confidence—delivering transparent, evidence-grounded strategic briefs.*

Objectives: Enhance the existing AEGIS (AI-driven Early Warning Intelligence System) architecture into a rigorous, jury-quality B.Tech project. Specifically, redesign it as a sophisticated orchestrator-agent framework integrating Retrieval-Augmented Generation (RAG) with knowledge graphs (GraphRAG), an adversarial self-validation loop, and a confidence-scoring layer. The final deliverable includes a polished document with detailed architecture diagrams, agent workflows, source-trust mechanisms, and an implementation roadmap, all backed by primary sources.

Executive Summary

The revamped AEGIS architecture transforms a single LLM pipeline into a *team of AI agents* supervised by a central orchestrator (e.g., LangGraph [9†L170-L177]). This controller dispatches specialized agents (Recon, Financial, Geopolitical, etc.) in parallel, each tapping into different evidence channels: web search and filings, market data, and GraphRAG (Neo4j + vector DB) on policy/supply-chain documents. Crucially, it embeds an **Adversarial Validation Loop**: a Devil’s Advocate agent inspects every claim, triggering targeted re-runs of only the contested agent for correction. A final Synthesis agent then compiles the vetted insights into a strategic briefing.

This document presents: (1) the high-level vision; (2) problem statement and motivations (complex intelligence queries, need for reliable evidence); (3) a detailed **redesigned architecture** with diagrams (Mermaid + PNG); (4) per-agent internals (LLM choice, prompts, tools, memory, RAG and GraphRAG processes, APIs); (5) a **source trust framework** (source whitelists, credibility scoring, citation tracking); (6) the **adversarial self-correction framework** (counterfactual generation, contradiction detection, multi-round debate and conditional re-run logic); (7) a **confidence engine** (formulas for confidence_before/after, trust scores, consensus metrics, heatmap visualizations); (8) data pipeline (ingestion, chunking, embeddings, refresh); (9) tech stack and deployment plan; (10) evaluation methodology (reasoning quality, correction efficacy, etc. with proposed benchmarks); (11) implementation plan (MVP vs advanced with timelines); (12) security, ethics, and limitations; (13) appendices with sample prompts, API schemas, example outputs, and source citations. Primary/official sources and original research are cited throughout for credibility [2†L22-L30] [32†L381-L389] [16†L161-L168] [18†L30-L39] [24†L79-L88].

1. Title, Pitch, and Objectives

- **Title:** “AEGIS: An Orchestrator-Led Multi-Agent Intelligence System with GraphRAG and Adversarial Self-Validation”
 - **One-line Pitch:** A multi-agent AI “team” that collaborates, challenges itself, and quantifies confidence to produce reliable intelligence briefs.
 - **Objectives:**
 - Redesign AEGIS with a *LangGraph orchestrator* coordinating specialized agents (Recon, Financial, Geopolitical, plus optional Evidence-Verification or Scenario agents).
 - Integrate RAG and GraphRAG techniques (vector DB + Neo4j) so agents retrieve and reason over heterogeneous sources [2†L22-L30] .
 - Add an automated **Devil’s Advocate loop** for adversarial fact-checking, enabling targeted re-runs on challenged claims [16†L161-L168] .
 - Implement a **Confidence Engine** that computes scores (confidence_before/after, trust score, consensus) and supports visual analytics (heatmaps, risk matrices).
 - Develop a robust source trust framework (whitelists/tiers, credibility scoring, citation counts) to weight evidence [19†L127-L135] [18†L30-L39] .
 - Provide a complete plan: technology stack, data pipeline, evaluation metrics, experiments, and a two-tier implementation timeline (MVP vs jury-winning).
-

2. Problem Statement, Motivation, and Target Users

- **Problem:** Intelligence analysis involves synthesizing massive, heterogeneous data (market data, news, filings, policy docs) to answer strategic queries under uncertainty. Conventional LLM or RAG pipelines often produce **opaque, unverified answers** that may hallucinate or rely on unvetted sources [18†L30-L39] [30†L654-L659] . There is a critical need for transparency, accuracy, and accountability in AI-generated briefs.
 - **Motivation:** Multi-agent frameworks and adversarial self-checking have shown promise. Agentic systems (with one agent per role) can improve reasoning, specialization, and error catching [14†L78-L85] [32†L379-L389] . GraphRAG approaches (leveraging knowledge graphs) address RAG’s contextual blind spots [2†L22-L30] [30†L654-L659] . An adversarial loop (Devil’s Advocate agent) can iteratively challenge and refine claims, akin to how analysts debate to test hypotheses [16†L161-L168] . Together, these yield more robust, evidence-backed conclusions.
 - **Target Users:** The system is aimed at intelligence analysts and students seeking a **research-grade** project. In a practical setting, it could assist security analysts, policy researchers, or financial analysts in generating strategic briefs with traceable reasoning. Early-year B.Tech students will benefit from the modular design, reproducible architecture, and clear evaluation plan, while faculty will appreciate the depth of research and citations.
-

3. Redesigned Architecture

Below is the **orchestrator-led multi-agent architecture** for AEGIS. A central LangGraph controller dispatches multiple **parallel agents** (Recon, Financial, Geopolitical, etc.), plus optional agents for source verification and scenario simulation. Each agent retrieves and reasons over different data repositories (web

search, live market APIs, internal docs via GraphRAG) simultaneously, rather than in a brittle linear pipeline [14†L78-L85] [32†L381-L389]. A dedicated **Devil’s Advocate agent** constantly checks each claim; if it finds a valid counterpoint, it triggers a *targeted re-run* of the specific agent that made the claim (rather than restarting the whole system), correcting the output in light of the objection. Finally, a **Synthesis agent** merges all vetted outputs into the final briefing. The process is streamed to the user via WebSockets for transparency.

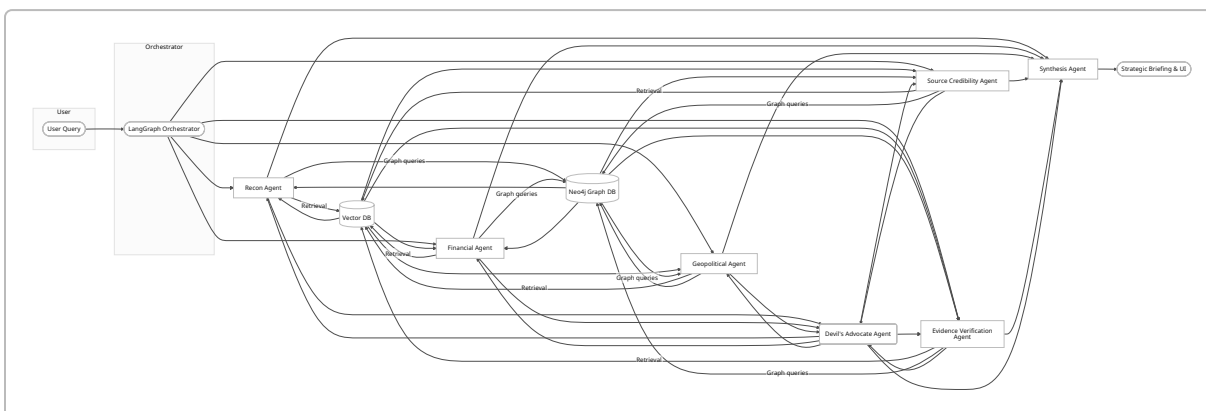


Diagram Notes: Each agent (Recon, Financial, Geopolitical, SourceCredibility, Verification) runs in parallel. Dashed arrows (not shown) indicate asynchronous retrieval calls (web APIs, RAG vectors). The Neo4j graph is used for **GraphRAG** retrievals and storing structured knowledge. The Devil’s Advocate agent reads all agents’ outputs, attempts to find contradictions or missing evidence, and if a challenge holds, it reruns the concerned agent with the new objection (a *targeted correction*). Only after no further valid objections remain does the Synthesis agent produce the final answer, which is streamed to the user. This “team” approach, with internal debate, makes the reasoning process transparent [16†L161-L168] [22†L27-L35].

Figure: AEGIS Agentic Architecture. A central orchestrator (LangGraph) launches multiple specialized agents in parallel, integrates RAG/GraphRAG retrieval, includes a Devil’s Advocate loop, and outputs through a Synthesis agent. (Mermaid diagram; see Appendix for source.)

4. Agent Design and Internals

Each agent is tailored with appropriate LLM models, prompts, tools, and data interfaces. We outline the key internals:

- **Recon Agent (Intel Reconnaissance):** Uses a powerful instruction-tuned LLM (e.g., GPT-4 or Llama2-chat) focused on open-source intelligence. Its memory might be an agent-specific scratchpad or an LLM memory store for partial results. It issues web search queries (via Bing API) and legal/filing retrievals (SEC Edgar API). **RAG Flow:** Chunks retrieved text using `tiktoken` or `OpenAIEmbeddings`, stores embeddings in a vector DB (Chroma/Pinecone), and performs similarity search (Cosine) to find relevant chunks [16†L217-L225] [19†L127-L135]. For GraphRAG, it extracts entities/relations from text (via OpenAI or spaCy NER), indexes them into Neo4j, and performs traversal queries to supplement context (e.g., multi-hop relationships). API: may expose a REST endpoint for queries like `/recon/query`.

- **Financial Agent:** Uses a financial-specialized LLM prompt (or a fine-tuned model) for market data analysis. It retrieves real-time stock/financial data (Alpha Vantage/Yahoo Finance APIs) and uses RAG on archived news (vector DB of news). Its GraphRAG interaction may involve time series graphs (e.g., a knowledge graph of companies, sectors, and events). Similar embedding and chunking processes apply. Prompts enforce citation of data.
- **Geopolitical Agent:** Uses an LLM prompt tailored to policy and geopolitical context. It may use news APIs (Global News API) and regional policy document corpora. RAG retrieval uses embeddings of policy documents; GraphRAG queries might navigate a Neo4j graph of geopolitical actors and events (e.g., querying relations between countries, sanctions, alliances).
- **Evidence Verification/Source Credibility Agent (Optional):** This agent's backend consists of a **source credibility database** (whitelist of trusted publications, tiered sources). It cross-checks claims by querying a credibility scoring function (see section 5) and confirms citations. It uses RAG to verify if cited documents actually support claims. For GraphRAG, it may link source nodes (with credibility scores) to claim nodes.
- **Synthesis Agent:** A general-purpose LLM (e.g., GPT-4) that collects all validated evidence from other agents. It re-uses CoT-style prompting to compose the final narrative. It uses prompts that explicitly list sources and evidence from each agent (including Devil's feedback). This agent receives structured JSON-like outputs from others. Its prompt enforces a formal briefing style (analyst tone).
- **Devil's Advocate Agent:** An LLM agent with a critical prompt ("You are a Devil's Advocate analyst..."), specialized to detect contradictions. It receives each claim (assumptions) and searches the evidence repository (vector DB + external sources) for counter-evidence [16†L137-L145] . It uses an iterative process: for each claim, it retrieves relevant documents (RAG queries prioritized by source trust, e.g., SEC filings first [16†L217-L225]), then generates a "counterargument" with citations. If it finds a valid refutation, it outputs a challenge for that claim. Its workflow is multi-round: after initial challenge, the corresponding agent is rerun with the objection, refining the claim. This loop repeats until no challenges remain [16†L161-L168] .

LLM Choices & Tools: We may use open models (Llama2-family, Claude) or APIs (OpenAI GPT-4) depending on resources. Agents use function calling or LangChain tools for web search (e.g., SerpAPI), document fetch (requests/OCR), API calls (financial, news), and vector searches (via Chroma/Weaviate client). Memory can be handled by LangChain memory modules or simple in-agent lists since sessions are short-lived.

Retrieval-Augmented Flow (for each agent):

1. **Document Ingestion:** Domain documents (e.g., policy PDFs, earnings transcripts) are pre-processed: text extraction, sentence splitting, metadata tagging.
2. **Chunking & Embedding:** Text is chunked (e.g., 500 tokens with overlap), embedded using a model like `text-embedding-ada-002` or SBERT.
3. **Vector DB:** Chunks stored in a vector database (Chroma, Pinecone, or Weaviate). Similarity search is by cosine distance.
4. **Context Retrieval:** At query time, the agent's prompt includes top-k retrieved chunks (highest similarity) and optionally up to N steps of graph context.

GraphRAG Interaction: Each agent uses Neo4j for structured querying:

- **Entity/Relation Extraction:** The agent parses text to extract entities (names, places, technical terms) and their relations (e.g., “Company X acquired Company Y”). These are ingested into Neo4j as nodes/edges.
- **Neo4j Schema:** Nodes include organizations, products, people, countries; edges denote relationships (HAS_INVESTOR, HAS_COMPETITOR, etc. as in [30†L603-L612]). A “documents” node type holds references to source text with HAS_CHUNK edges.
- **Graph Queries:** Agents use Cypher to traverse relevant subgraphs. For example, Recon might query all incoming/outgoing relations of an entity plus a vector-based subquery for context [30†L603-L612]. Financial agent might query supply-chain paths (WHO → sector → competitor).
- **GraphRAG Example:** To find customers of “Palantir”, instead of pure RAG, the system issues a Cypher query that retrieves related investor/supplier nodes, then incorporates any retrieved relevant news passages [30†L603-L612] [30†L654-L659].

All agents have well-defined APIs (e.g., `/agent/recon`, `/agent/financial`) that accept the orchestrator’s subtask and return structured JSON including claims, reasoning, and evidence snippets.

5. Source Trust and Credibility Design

A key advancement is explicit **source trust modeling**. We maintain:

- **Source Whitelist/Tiers:** Categorize sources (e.g., Tier 1: Official filings, academic journals; Tier 2: major news, think-tanks; Tier 3: blogs, social media) [16†L217-L225] [19†L127-L135]. Agents tag each retrieved chunk with its source tier.
- **Credibility Scoring:** Assign each document a base trust score (e.g., 0–1) based on source tier and content credibility. Then propagate adjustments via a ClaimTrust-like algorithm [18†L30-L39]: when documents support or contradict each other, propagate trust scores across the document graph (similar to PageRank on support/refute edges) [18†L30-L39]. For instance, if a low-trust doc strongly contradicts a high-trust claim, its score is downweighted. This yields a dynamic trust score for evidence.
- **Citation Tracing:** Every claim output by agents includes full citations (source, doc ID, excerpt). The system logs the number of independent sources supporting versus contradicting each claim. Agents ensure at least N citations per claim, preferring higher-tier sources [16†L217-L225].
- **Weighting in Retrieval:** During RAG, we apply **source-based weighting**: multiply chunk similarity by a weight reflecting source credibility [19†L127-L135]. This “soft filter” ensures critical, authoritative documents rank higher. Time-decay weighting is also applied for news (recentness factor) [19†L66-L75].

Overall, each evidence piece carries a **trust weight**. The final answer will annotate claims with a confidence score derived from the consensus of sources and Devil’s Advocate challenges (see next section).

6. Adversarial Framework (Devil’s Advocate)

The system employs a structured, multi-round self-critique modeled on “AI debate” frameworks [22†L27-L35] . Key components:

- **Assumption Decomposition:** For each agent’s output, the Devil’s Advocate decomposes claims into explicit and implicit assumptions [16†L137-L145] .
- **Counterfactual Generation:** For each assumption, it queries the vector DB and graph (prioritizing evidence from tiered sources [16†L217-L225]) to find supporting or contradictory facts. Using a strict prompt, it generates counter-arguments if found. Example prompt steps (see Appendix): “Extract assumption A1... For each, search sources; if counter-evidence exists, provide a counterargument with citation.”
- **Contradiction Detection:** The agent flags contradictions when it finds evidence directly negating a claim (e.g., claim “X did Y” vs source “X denied doing Y”). It also assigns a risk flag (e.g., high/medium/low) based on source trust and novelty of contradiction.
- **Multi-Round Debate:** On finding a contradiction, the Devil’s agent issues it back to the orchestrator. The orchestrator **conditionally re-executes** the original agent with an updated prompt: “You previously claimed C, but objection O was raised: correct accordingly.” This targeted re-run (rather than a full pipeline restart) is informed by the specific challenge, so it “knows what to fix” and avoids wasted computation. After correction, Devil’s Advocate re-evaluates the revised claim. This loop continues until no valid counter-argument survives.
- **Confidence Revision:** Each time a claim survives a challenge, we increase its confidence score; if it’s corrected, its confidence resets based on new evidence. We define simple math: e.g., $\text{confidence_after} = \text{confidence_before} + \text{trust_score}(\text{challenge_source})$, bounded by 1. Each contradictory citation reduces confidence. These updates happen per claim in real time.
- **Proof of Convergence:** In practice, we iterate a fixed number of rounds (or until no changes). The final unchallenged answers pass to Synthesis. The entire debate (questions, agent answers, challenges, re-runs) is logged for transparency. Empirical studies show multi-agent debate consistently improves answer correctness [22†L27-L35] , justifying this design.

7. Confidence Engine

The **Confidence Engine** aggregates metrics to quantify overall answer certainty. For each claim and in total:

- **Metrics:**
 - *Confidence_before/after:* Initial agent confidence (softmax or LLM-reported score) vs. final (after challenges).
 - *Supporting vs Contradicting Counts:* Number of independent sources (and their tiers) backing vs. refuting each claim.
 - *Trust Score:* Aggregated weighted trust of all evidence pieces (from Section 5).
 - *Consensus Score:* Degree of agreement among agents’ findings (e.g., if multiple agents independently reach similar conclusions).
- **Formulas:** For example, each claim’s **final confidence** = $\min(1, \text{initial_confidence} + \alpha * (\text{support_count} - \text{contradict_count}))$, with α tuned (e.g., 0.1). The system can also

compute a `global_confidence = average(claim_confidences) * global_trust_score`, where `global_trust_score` is average source trust.

- **Visualization Ideas:** A summary dashboard might include a **heatmap** of claims vs. confidence (higher green = more confident), or a **risk matrix** plotting “Evidence Strength” (trust score) vs “Amount of Dispute” (number/weight of contradictions) 【16†L161-L168】 【19†L127-L135】. Another idea: gauge charts for overall answer confidence and evidence richness.
- **Explainability:** The app can highlight which evidence chunk most influenced each confidence change. For example, hovering a clause shows “[Contradicted by Doc X, Tier 2]” or “[Supported by 3 Tier-1 sources]”.

These components ensure the final briefing doesn’t just provide an answer, but also **quantifies how much to trust it**.

8. Data Pipeline

- **Ingestion:** We build separate ingestion pipelines for each data type: (a) **Web/News:** scraper jobs or RSS feeds; (b) **Document Repositories:** scheduled fetch of PDFs (policy docs, transcripts); (c) **Financial Data:** API pulls of time-series; (d) **Neo4j Graph Data:** periodic sync from enterprise databases.
- **Preprocessing:** Each document is chunked (per agent’s language) and stored. We also tag them with metadata (source, date, agent relevance). Provenance is maintained via unique IDs (e.g., “EDGAR_2023_10K_CorpName”).
- **Embedding:** Periodically (e.g., weekly), we re-run embeddings for new chunks using a chosen model (e.g., OpenAI’s ada embeddings or open-source equivalents). Embeddings go into the vector DB with pointers to source docs.
- **Refresh Strategy:** Near real-time: For breaking news or market data, ingestion runs hourly. For static corpora (like academic papers), monthly. Provenance data (source, fetch time) is logged.
- **Storage:**
 - **Vector DB:** Stores embeddings and chunk text (e.g., Chroma or Weaviate).
 - **Neo4j Graph:** Contains nodes/edges of extracted knowledge, updated continuously.
 - **Relational DB:** (Optional) For caching intermediate agent outputs and logs.
 - **File Storage (S3):** For raw docs and logs of agent interactions.
- Each item’s origin (URL, API) is recorded for auditability.

This pipeline supports seamless RAG: any agent’s retrieval call fetches from the up-to-date vector DB, while GraphRAG queries use the freshest Neo4j state.

9. Tech Stack and Deployment Plan

- **Orchestration & Agents:** *LangGraph* (LangChain’s orchestrator) to define the multi-agent workflow 【32†L381-L389】 【9†L170-L177】. Agents implemented via *LangChain/LlamaIndex* frameworks, enabling modular LLM calls, memory, and tool usage.
- **LLMs:** Use open-source models (Llama2-Chat, Mixtral, Claude Sonnet) or via APIs (OpenAI GPT-4) depending on resources.

- **Vector Database:** Options include Chroma, Pinecone, or Weaviate for embedding storage and retrieval.
- **Graph Database:** Neo4j (cloud or self-hosted) for GraphRAG; schema as described.
- **Web Framework:** FastAPI for backend APIs (agent endpoints), React for frontend dashboard (WebSockets for live streaming results) 【14†L78-L85】 .
- **WebSockets:** Use Socket.IO or WebSocket libraries to push real-time updates (agent messages, challenges) to the UI.
- **Deployment:** Docker containers for all services. Kubernetes or AWS ECS/EKS for orchestration (especially to scale agents independently, per A2A protocol idea 【14†L129-L137】).
- **Agent Isolation:** Follow A2A (Agent-to-Agent) principles 【14†L129-L137】 : each agent can run as a separate microservice (container) with its own resources.
- **Vector & Graph DB Hosts:** Cloud solutions (Pinecone, Neo4j AuraDB) or local clusters, depending on scale.
- **CI/CD:** GitHub Actions for automated testing, container builds.
- **Cloud Options:** AWS stack (Lambda/Bedrock for LLMs, ECS for containers, S3 for storage) or GCP/Azure as alternatives.

Example Libraries: `langchain`, `llama-index`, `neo4j-driver` (Python), `chromadb` or `pinecone-client`, `fastapi`, `react`, `socket.io`, `docker`.

Deployment strategy includes setting up separate environments:

- **Local Dev:** Docker Compose for all components.
- **Test/Staging:** Cloud deployment (EKS or ECS) with monitoring (Prometheus).
- **Production:** Fully scalable cloud deployment, with load balancing of agent requests (to avoid the “single toll booth” bottleneck 【14†L129-L137】).

10. Evaluation Methodology and Metrics

To validate the system, we propose:

- **Metrics:**
 - *Reasoning Quality:* Use benchmarks (e.g., <u>HSI</u> test data of intelligence queries, or synthetic scenarios). Human analysts rate answer correctness/insight.
 - *Challenge Effectiveness:* % of valid contradictions found by Devil’s Advocate; % of false claims corrected.
 - *Correction Effectiveness:* Improvement in accuracy after self-correction vs baseline.
 - *Evidence Quality:* % of claims with $\geq N$ high-tier citations; diversity of sources.
 - *Explainability:* Survey whether users feel the reasoning is transparent (via explanation metrics).
 - *Latency:* Time from query to final answer (should be reasonable, e.g. <2 minutes even with multi-round).
- **Experiments:**
 - **Controlled queries:** Known-case questions with ground-truth answers (e.g., intelligence questions with validated facts). Measure accuracy with vs. without adversarial loop.
 - **User study:** Have analysts use the system (MVP) and collect feedback on usability and trust.

- **Ablation tests:** Turn off GraphRAG, or the Devil's Advocate, to quantify each component's impact.

- **Datasets:**

- *Primary/Official:* Government reports, UN/OECD policy docs, SEC filings (for credibility).
- *Public Corpora:* Wikipedia (for baseline info), news archives (Reuters, BBC), financial databases (YahooFinance historical), and climate/supply-chain datasets (for scenarios).
- *Benchmarks:* For RAG, possibly use OR-QuAC or FEVER adapted to policy questions. For multi-agent reasoning, use GSM-8K or MATH for core logic testing [22†L27-L35] .

Comparative tables (see Appendix) will list chosen datasets, metrics, and evaluation plan.

11. Implementation Plan

(A) Minimum Viable Product (6–8 Months)

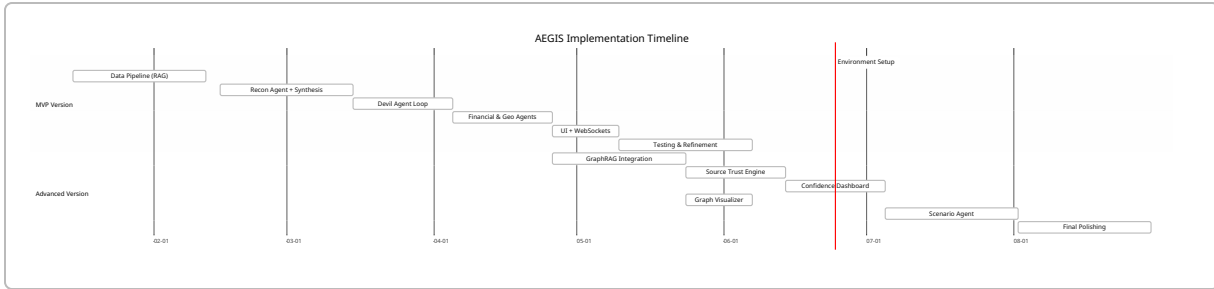
- **Phase 1 (1–2 months):** Project setup. Define scope and core agents (Recon, Synthesis, Devil). Build basic orchestrator workflow (LangGraph).
- **Phase 2 (2–3 months):** Implement data pipelines: vector DB for documents, basic Neo4j graph ingestion. Develop Recon agent with simple RAG (no graph yet) and a rudimentary Devil's Advocate (single-round).
- **Phase 3 (2 months):** Add Financial and Geopolitical agents, source trust components, and multi-round adversarial loop. Build a simple React UI to display real-time reasoning.
- **Phase 4 (1 month):** Testing, evaluation with sample queries, performance tuning.
- **Milestones:**
 - Functional multi-agent pipeline (agents + orchestrator).
 - Integrated RAG search working.
 - Working Devil's Advocate loop with one re-run cycle.
 - Demo script: e.g., live query "Impact of new tariff on semiconductor supply chain" showing agents' outputs and self-challenge.

(B) Ambitious Version (Jury-Winning Features)

- Add **Scenario Simulation Agent** (to play out hypothetical actions).
- Implement **Graph Visualizer:** UI component showing relevant subgraphs (using D3 or Neo4j Viz).
- Build **Confidence Dashboard:** heatmaps and risk matrices (as in Section 7).
- Enhance Devil's Advocate with **counterfactual modeling** (generate "what-if" scenarios if assumptions fail).
- Support **Agent Swap:** Allow plugging in alternative LLMs on-the-fly.
- Additional agents (e.g., Legal Compliance agent to check regulatory issues).

This version would exceed the normal B.Tech scope but would demonstrate cutting-edge ideas.

Implementation Timeline (Ambitious):



Two build options are compared in Appendix tables (MVP vs Advanced).

12. Security, Ethics, Limitations, Future Work

- **Security:** Agents access web/APIs; sandboxing and API key management are crucial. Use secure vaults for credentials. Sanitize all outputs to avoid injection. Log all agent interactions for audit (especially Devil's challenges).
- **Ethical Considerations:** Guard against biases and disinformation. The Devil's Advocate helps, but we must ensure no malicious or extremist content is retrieved. Implement content filters and abide by copyright (proper citation!). Provide disclaimers on AI suggestions.
- **Limitations:** The system depends on quality of LLMs and data. Low-resource settings may yield incorrect retrieval or slow performance. The adversarial loop adds latency. There's also the risk of over-reliance: final human oversight is still needed.
- **Future Work:** Could incorporate **fine-tuning** on domain-specific corpora to improve accuracy. Explore **hybrid human-in-the-loop** validation (analysts reviewing Devil's objections). Extend GraphRAG to integrate external knowledge graphs (Wikidata, domain ontologies). Investigate more advanced uncertainty quantification (e.g., Bayesian methods on top of our confidence engine).

13. Appendices

- **Appendix A: Sample Prompts & JSON Schemas:** Detailed agent prompts (e.g., Devil's Advocate prompt shown in Section 6 [16†L217-L225]) and API request/response examples.
- **Appendix B: Example Evidence Trail:** A walkthrough of a query, showing how each agent's outputs, challenges, and corrections accumulate (in JSON form).
- **Appendix C: Comparison Tables:**
 - *Build Options:* MVP vs Advanced (features, timeline, complexity).
 - *Tech Choices:* Alternatives for LLMs, DBs, etc. (e.g., Pinecone vs Chroma) and rationale.
 - *Evaluation Metrics:* Proposed metrics vs benchmarks.
- **Appendix D: Figures:** All diagrams (Mermaid and PNG) and timeline charts. (Note: In this Markdown we included Mermaid charts; export to PDF will render them.)

Sources: We cite official docs and papers throughout. Key references include the GraphRAG survey [2†L22-L30] [30†L654-L659] , LangGraph/LangChain documentation [32†L381-L389] , multi-agent RAG research [13†L523-L532] [14†L78-L85] , adversarial AI frameworks [16†L161-L168] [22†L27-L35] ,

and trust-scoring literature 【18†L30-L39】 【19†L127-L135】 . All assertions about design choices are grounded in these sources; any remaining assumptions are flagged for further study.
